

Product Requirements Document (PRD): Real-Time Emergency Voice AI System

High-stakes, ultra-low-latency conversational AI for first responders and healthcare field workers

1. Executive Summary

This project aims to build a high-stakes, ultra-low-latency Real-Time Voice and Conversational AI system designed for first responders and healthcare field workers. The system provides hands-free, AI-guided medical assistance that intelligently differentiates between varying levels of trauma. By leveraging Moss for sub-10ms context retrieval and WebRTC for seamless human-in-the-loop escalation, the platform ensures that responders receive immediate, protocol-accurate guidance while bridging in medical professionals for critical life-saving interventions.

2. Problem Statement

Field workers and first responders often operate in high-stress environments where manual data entry or "walkie-talkie" style communication is impossible or too slow. Existing voice assistants lack the medical context, emotional intelligence (prosody), and sub-millisecond state awareness required to manage complex medical emergencies. There is a critical need for a system that can:

- Provide instant, hands-free medical guidance.
- Maintain a "live" state of the emergency (e.g., time since last breath).
- Automatically escalate to human doctors when AI thresholds are exceeded.

3. Goals & Objectives

- Sub-10ms Context Retrieval: Utilize Moss to ensure the AI has instant access to session facts.
- Adaptive Triage: Implement logic to handle scenarios ranging from low-criticality (simple cuts) to high-criticality (drowning).
- Seamless Handover: Enable zero-delay transitions from AI-led guidance to human-led intervention via WebRTC.
- Automated Documentation: Generate structured medical SOAP notes immediately following an incident.
- Reliability: Ensure system resilience through persistent session caching and redundancy.

4. Target Users / Stakeholders

- First Responders/Field Workers: Primary users requiring hands-free guidance.

- Medical Professionals (Doctors/Dispatchers): Secondary users who monitor or take over high-criticality calls.
- Regulatory/Audit Bodies: Stakeholders requiring precise transcripts and action logs for compliance.
- Product/Engineering Teams: Users of the Scenario Test Harness for system validation.

5. Functional Requirements

5.1 Real-Time Voice Interaction

- Hands-Free Operation: Full-duplex audio streaming via WebRTC.
- STT with Prosody: Real-time transcription including emotional tonality analysis (panic, urgency).
- Ultra-Low Latency TTS: Human-like, calm voice instructions delivered back to the responder.

5.2 Intelligence & Context Management

- State Awareness: Track medical vitals and actions (e.g., "tourniquet applied at 10:05 AM") using Moss.
- Protocol Retrieval: RAG-based querying of verified medical knowledge bases (Qdrant).
- Diarization: Distinguish between the responder, the patient, and the doctor.

5.3 Escalation & Human-in-the-Loop

- Criticality Thresholds: Automatic detection of Level 1-5 emergencies.
- Doctor Bridge: Immediate WebRTC audio bridging for medical professionals.
- Live Dashboard: Real-time data visualization for doctors joining a call, showing the AI's current state and history.

5.4 Post-Incident Reporting

- SOAP Note Generation: Automated synthesis of transcripts and Moss context into structured medical reports.
- Synchronized Playback: Interface to review audio synced with time-stamped transcripts and emotional metadata.

5.5 Testing & Simulation

- Scenario Test Harness: Programmatic simulation of responders and doctors with variable dialogue and timing.
- Validation: Automated checking of Moss state updates and escalation trigger accuracy.

6. Non-Functional Requirements

- Performance: Sub-10ms retrieval for Moss context; sub-500ms end-to-end voice round-trip.
- Reliability: 99.9% uptime for the Voice Gateway; Redis-backed session persistence for failover.
- Scalability: Support for concurrent emergency sessions using SFU (Selective Forwarding Unit) architecture.
- Safety: Pydantic-enforced data schemas to ensure medical instruction accuracy.

7. System Architecture Overview

The system is divided into seven functional layers:

1. Client Layer: Mobile app (Responder) and Web Dashboard (Doctor).

2. Edge & Media Layer: Voice Gateway and WebRTC Media Server for audio routing.
3. Intelligence Layer: STT, TTS, and the LangGraph-based Orchestrator.
4. Context & Knowledge Layer: Moss (Hot context), Qdrant (Protocols), and Redis (Backup).
5. Escalation Layer: Triage logic, Alert Queues, and Notification services.
6. Audit & Review Layer: Time-series databases and SOAP note generation.
7. Testing & Resilience Layer: Scenario Test Harness and Resilience Roadmap.

8. Tech Stack

- Languages: Python (Orchestrator, FastAPI), Rust (Moss, Media Server), Node.js (Notifications), React Native (Mobile), React/Next.js (Dashboard).
- AI/ML: Deepgram (STT), ElevenLabs Turbo v2.5 (TTS), LangGraph/Pydantic AI (Orchestration), OpenAI GPT-4o (Summarization).
- Data/Storage: Moss Context Engine, Qdrant (Vector DB), Redis Stack (Cache), PostgreSQL/TimescaleDB (Audit), Amazon S3/Minio (Audio).
- Communication: WebRTC (LiveKit), WebSockets, gRPC, RabbitMQ/Amazon SQS.

9. Data Requirements

- Moss Context: Key-value store for immediate session facts (e.g., `cpr_start_time`, `bleeding_controlled`).
- Time-Series Transcripts: Millisecond-precision logs of dialogue and prosody markers.
- Medical Protocols: Vectorized medical manuals for RAG.
- Session Cache: Redis-JSON snapshots for state recovery.

10. API Specifications

- WebSocket `/v1/emergency/stream`: Bi-directional audio and event stream.
- gRPC `MossService`: High-speed internal interface for context read/writes.
- REST `/v1/audit/summary/{session_id}`: Retrieves generated SOAP notes.
- REST `/v1/session/playback/{session_id}`: Provides signed URLs for audio and synced transcript JSON.

11. Security Requirements

- Authentication: Secure token-based access for responders and doctors.
- Data Protection: Encryption at rest for audio and transcripts; HIPAA-compliant data handling protocols.
- Authorization: Role-based access control (RBAC) for the Medical Personnel Dashboard.

12. Deployment & Infrastructure

- Containerization: Docker-based microservices.
- Orchestration: Kubernetes for scaling the Orchestrator and Media Servers.
- Cloud: AWS (API Gateway, S3, SQS).
- CI/CD: Automated testing via the Scenario Test Harness before deployment.

13. Success Metrics

- Context Latency: 100% of Moss retrievals under 10ms.
- Escalation Accuracy: 0% false negatives for Level 5 (Critical) scenarios in testing.
- Documentation Speed: SOAP notes generated within 30 seconds of session termination.
- User Sentiment: Calmness and clarity ratings of TTS instructions.

14. Timeline & Milestones

1. Phase 1: Core Voice Loop: STT -> Orchestrator -> Moss -> TTS integration.
2. Phase 2: Medical RAG & Triage: Qdrant integration and escalation logic.
3. Phase 3: Human-in-the-Loop: WebRTC Media Server and Doctor Dashboard.
4. Phase 4: Audit & Summary: TimescaleDB logging and SOAP note generation.
5. Phase 5: Validation: Full-scale simulation using the Scenario Test Harness.

15. Open Questions & Risks

- Network Degradation (TODO): How the system handles 3G/Low-bandwidth environments in the field.
- Doctor Drop-off (TODO): Logic for the AI to resume the lead if the doctor loses connection.
- Late Join (TODO): Moss-driven "catch-up" summaries for doctors joining mid-incident.
- Chaos Engineering: Footnote for future implementation to test service failures during active calls.